CPU Scheduling: FCFS and Round Robin

First Come First Served (FCFS) is the simplest CPU scheduling algorithm. Processes are placed in a ready queue in the order they arrive, and the CPU is given to the process at the head of the queue. FCFS is non-preemptive: once a process gets the CPU it keeps it until it finishes or blocks for I/O.

FCFS can produce a long average waiting time. When one long process arrives before several short ones, the short processes wait behind it. This is called the convoy effect.

Round Robin (RR) is designed for time-sharing systems. Each process receives a small unit of CPU time called a time quantum. When the quantum expires, the process is preempted and moved to the back of the ready queue. If the quantum is too large, RR behaves like FCFS; if it is too small, context switching overhead dominates.

Priority Scheduling and Starvation

In Priority Scheduling every process is assigned a priority and the CPU is allocated to the process with the highest priority. Processes with equal priority are scheduled in FCFS order. Priority scheduling can be preemptive or non-preemptive.

The main problem with priority scheduling is starvation, also called indefinite blocking. In a busy system a steady stream of high-priority processes can keep the CPU away from the lower ones. Low-priority processes may wait indefinitely and never get to run.

Priorities can be defined internally, for example from memory requirements or the ratio of I/O burst to CPU burst, or externally, for example from the importance of the user or the process.

Aging

Aging is a technique that solves the starvation problem in priority scheduling. The priority of a process that has been waiting in the ready queue for a long time is gradually increased as time passes.

For example, the scheduler may raise the priority of every waiting process by one level every 15 minutes. Even a process that starts with the lowest priority will eventually reach a priority high enough to be selected, so it is guaranteed to execute.

Aging therefore trades a small amount of scheduling precision for fairness. Multilevel feedback queues use the same idea: a process that waits too long in a low-priority queue is moved up to a higher-priority queue.